Anexo 22 — Evaluacion multi-tarea (exploratorio)

Archivo: Scripts/eval_multitask.py

Para que sirvio:
Evaluacion del entorno multi-tarea exploratorio. No formo parte del pipeline final.

```
"""Evaluacion de una policy multi-task v14 (DUMMultitaskEnv).

Por cada una de las 7 combinaciones de flags activas (descartando [0,0,0]) corre
N episodios deterministicos, mide:
    - mean_head_err   (deg)   — promedio de theta_deg cuando flag_track=1
    - mean_wave_err           — promedio de ||arm_qpos - ref|| cuando flag_wave=1
    - mean_grip_err   (m)     — promedio de |slider - target| cuando flag_grip=1
    - mean_reward             — reward total promedio por episodio

Imprime una tabla con media±std por combo. Opcionalmente graba un video con
4 segmentos forzando flags: [1,0,0], [0,1,0], [0,0,1], [1,1,1].

Uso:
    python Scripts/eval_multitask.py runs/ppo_dum_v14/final.zip
    python Scripts/eval_multitask.py runs/ppo_dum_v14/final.zip --video out.mp4
    python Scripts/eval_multitask.py runs/ppo_dum_v14/final.zip --video  # auto-naming
"""

import argparse
import re
import sys
from itertools import product
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))

import numpy as np
import mujoco
from stable_baselines3 import PPO

from rl_env_multitask import DUMMultitaskEnv


ALL_FLAG_COMBOS = [
    np.array(c, dtype=np.float32)
    for c in product([0.0, 1.0], repeat=3)
    if sum(c) > 0
]  # 7 combos


def run_episode_with_flags(model, env, flags, seed, deterministic=True):
    """Corre un episodio forzando self._flags al combo dado. Devuelve metricas."""
    obs, _ = env.reset(seed=seed)
    # Override hard del estado interno: pisamos los flags muestreados.
    env.unwrapped._flags = flags.astype(np.float32).copy()
    # Re-build obs para reflejar el override.
    obs = env.unwrapped._get_obs()

    done, trunc = False, False
    ep_rew = 0.0
    head_errs = []
    wave_errs = []
    grip_errs = []

    while not (done or trunc):
        action, _ = model.predict(obs, deterministic=deterministic)
        obs, r, done, trunc, info = env.step(action)
        ep_rew += float(r)
        if flags[0] > 0.5:
            head_errs.append(info["theta_deg"])
        if flags[1] > 0.5:
            wave_errs.append(info["arm_err_norm"])
        if flags[2] > 0.5:
            grip_errs.append(abs(info["slider_q"] - info["grip_target"]))

    return dict(
        ep_rew=ep_rew,
        head_err=np.mean(head_errs) if head_errs else np.nan,
        wave_err=np.mean(wave_errs) if wave_errs else np.nan,
        grip_err=np.mean(grip_errs) if grip_errs else np.nan,
    )


def evaluate_all_combos(model, env, n_episodes=20, deterministic=True):
    """Para cada combo de flags, corre n_episodes y agrega estadisticas."""
    rows = []
```

```
    for combo in ALL_FLAG_COMBOS:
        per_ep = []
        for ep in range(n_episodes):
            seed = int(1000 * sum(combo) + ep)
            m = run_episode_with_flags(model, env, combo, seed=seed,
                                       deterministic=deterministic)
            per_ep.append(m)
        rew = np.array([x["ep_rew"] for x in per_ep])
        head = np.array([x["head_err"] for x in per_ep])
        wave = np.array([x["wave_err"] for x in per_ep])
        grip = np.array([x["grip_err"] for x in per_ep])
        rows.append(
            dict(
                flags=combo,
                ep_rew_mean=float(np.mean(rew)),
                ep_rew_std=float(np.std(rew)),
                head_err_mean=float(np.nanmean(head)) if not np.all(np.isnan(head)) else np.nan,
                head_err_std=float(np.nanstd(head)) if not np.all(np.isnan(head)) else np.nan,
                wave_err_mean=float(np.nanmean(wave)) if not np.all(np.isnan(wave)) else np.nan,
                wave_err_std=float(np.nanstd(wave)) if not np.all(np.isnan(wave)) else np.nan,
                grip_err_mean=float(np.nanmean(grip)) if not np.all(np.isnan(grip)) else np.nan,
                grip_err_std=float(np.nanstd(grip)) if not np.all(np.isnan(grip)) else np.nan,
            )
        )
    return rows


def print_table(rows):
    header = (
        f"{'flags':>10s}  {'reward':>16s}  {'head_err(deg)':>16s}  "
        f"{'wave_err':>16s}  {'grip_err(m)':>18s}"
    )
    print(header)
    print("-" * len(header))
    for r in rows:
        flag_str = "[{:.0f},{:.0f},{:.0f}]".format(*r["flags"])

        def fmt(mean, std, prec=2):
            if np.isnan(mean):
                return f"{'—':>16s}"
            return f"{mean:+8.{prec}f} ± {std:6.{prec}f}"

        print(
            f"{flag_str:>10s}  "
            f"{r['ep_rew_mean']:+8.2f} ± {r['ep_rew_std']:6.2f}  "
            f"{fmt(r['head_err_mean'], r['head_err_std'])}  "
            f"{fmt(r['wave_err_mean'], r['wave_err_std'], prec=3)}  "
            f"{fmt(r['grip_err_mean'], r['grip_err_std'], prec=4)}"
        )


def record_video(model, env, out_path, fps=50, model_path="", seconds_per_seg=5.0):
    """Graba un video con 4 segmentos forzando flags fijos."""
    import imageio
    from PIL import Image, ImageDraw, ImageFont

    renderer = mujoco.Renderer(env.model, height=480, width=480)

    try:
        font = ImageFont.truetype("arial.ttf", 13)
        font_small = ImageFont.truetype("arial.ttf", 11)
    except Exception:
        font = ImageFont.load_default()
        font_small = ImageFont.load_default()

    model_name = Path(model_path).name if model_path else "unknown"
    try:
        total_steps = int(model.num_timesteps)
    except Exception:
        total_steps = "?"

    segments = [
        ("track only", np.array([1, 0, 0], dtype=np.float32)),
        ("wave only", np.array([0, 1, 0], dtype=np.float32)),
        ("grip only", np.array([0, 0, 1], dtype=np.float32)),
        ("all on", np.array([1, 1, 1], dtype=np.float32)),
    ]

    steps_per_seg = int(round(seconds_per_seg * fps))
    frames = []

    for seg_idx, (seg_name, flags) in enumerate(segments):
        obs, _ = env.reset(seed=42 + seg_idx)
        env.unwrapped._flags = flags.copy()
        obs = env.unwrapped._get_obs()
```

```
|
|         ep_reward = 0.0
|         for step_idx in range(steps_per_seg):
|             action, _ = model.predict(obs, deterministic=True)
|             obs, r, done, trunc, info = env.step(action)
|             ep_reward += float(r)
|
|             renderer.update_scene(env.data, camera=-1)
|             frame = renderer.render()
|             img = Image.fromarray(frame)
|             draw = ImageDraw.Draw(img)
|
|             header_lines = [
|                 f"model:   {model_name}",
|                 f"trained: {total_steps:,} steps",
|                 f"segment: {seg_name}  flags={flags.astype(int).tolist()}",
|             ]
|             y = 6
|             for line in header_lines:
|                 draw.text((6, y), line, fill=(230, 230, 230), font=font_small)
|                 y += 13
|
|             ep_lines = [
|                 f"step {step_idx + 1:3d}/{steps_per_seg}",
|                 f"theta = {info['theta_deg']:5.1f} deg",
|                 f"arm_err = {info['arm_err_norm']:5.3f}",
|                 f"grip|err|= {abs(info['slider_q'] - info['grip_target']):5.4f}",
|                 f"reward = {ep_reward:+7.2f}",
|             ]
|             y = 6
|             for line in ep_lines:
|                 tw = draw.textlength(line, font=font)
|                 draw.text(
|                     (img.width - tw - 6, y),
|                     line,
|                     fill=(255, 240, 100),
|                     font=font,
|                 )
|                 y += 14
|
|             frames.append(np.array(img))
|
|             if done or trunc:
|                 # Si el episodio se trunco (500 steps) y aun nos sobran frames del
|                 # segmento, reseteamos manteniendo los flags.
|                 obs, _ = env.reset(seed=42 + seg_idx + 100)
|                 env.unwrapped._flags = flags.copy()
|                 obs = env.unwrapped._get_obs()
|                 ep_reward = 0.0
|
|     imageio.mimsave(out_path, frames, fps=fps, quality=8)
|     print(f"Video guardado en {out_path}  ({len(frames)} frames @ {fps} fps)")
|
|
| def main():
|     p = argparse.ArgumentParser()
|     p.add_argument("model_path", type=str)
|     p.add_argument("--episodes", type=int, default=20,
|                    help="Episodios por cada combinacion de flags (7 combos).")
|     p.add_argument("--video", type=str, nargs="?", const="AUTO", default=None,
|                    help="Si se pasa '--video' sin valor, autogenera "
|                         "eval_multitask_v<N>.mp4 al lado del modelo.")
|     p.add_argument("--stochastic", action="store_true")
|     args = p.parse_args()
|
|     if args.video == "AUTO":
|         run_dir = Path(args.model_path).parent
|         m = re.search(r"v(\d+)", run_dir.name)
|         num = m.group(1) if m else "x"
|         args.video = str(run_dir / f"eval_multitask_v{num}.mp4")
|         print(f"Video auto-naming: {args.video}")
|
|     env = DUMMultitaskEnv()
|     model = PPO.load(args.model_path)
|     print(f"Loaded: {args.model_path}")
|     print()
|
|     rows = evaluate_all_combos(
|         model, env, n_episodes=args.episodes,
|         deterministic=not args.stochastic,
|     )
|     print_table(rows)
|
|     if args.video:
|         print()
```

```
247 |         record_video(model, env, args.video, model_path=args.model_path)
248 |
249 |
250 | if __name__ == "__main__":
251 |     main()
252 |
```